LAPORAN PRAKTIKUM

POSTTEST 5

ORGANISASI DAN ARSITEKTUR KOMPUTER



Disusun oleh:

M. Rasya Rifaldy Mas'ud (2609106073)

Kelas C1'26

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

Soal no 1

Tabel 1.1 Observasi Instruksi R-type dan I-type

Instruksi	PC	RegWrite	ALUSrc	ALU in 1	ALU in 2	Hasil ALU
Addi t0, zero, 18	0x00000000	1	1	0	18	18
Addi t1, zero, 6	0x00000004	1	1	0	6	6
Sub t2, t0, t1	0x00000008	1	0	18	6	12
And t3, t0, t1	0x0000000C	1	0	18	6	2
Or t4, t0, t1	0x00000010	1	0	18	6	22
Nilai akhir Register	T2	Desimal: 12		Heksadesimal: 0x0000000C		
	T3	Desimal: 2		Heksadesimal: 0x00000002		
	T4	Desimal: 22		Heksadesimal: 0x00000016		

Tabel 1.2 Field Pembeda Instruksi R-type

Instruksi	Funct3	Funct7	Opcode
Sub t2, t0, t1	000	0100000	0110011
And t2, t0, t1	111	0000000	0110011
Or t2, t0, t1	110	0000000	0110011

A. Mengapa ALUSrc pada addi berbeda dengan sub? :

   ALUSrc pada addi bernilai 1 karena operand kedua ALU diambil dari nilai konstanta yang telah diperluas oleh Immediate Generator. Sedangkan pada sub, ALUSrc bernilai 0 karena operand kedua ALU diambil langsung dari isi register kedua (rs2 / t1).

B. Field pembeda sub dari add :

   Keduanya memiliki opcode (0110011) dan funct3 (000) yang identik. Field yang membedakan keduanya adalah funct7, di mana add bernilai 0000000 sedangkan sub bernilai 0100000.

Soal 2: Analisis Datapath Load, Store, dan Branch

Tabel 2.1 Observasi Instruksi Load & Store

Instruksi	RegWrite	memReadd	memWrite	ALU in 1	ALU in 2	Data
Sw t0, 4(sp)	0	0	1	0x7ffffff0	4	77
Lw t0, 4(sp)	1	1	0	0x7ffffff0	4	77
Nilai akhir Regrister	T2	Desimal: 0		Heksadesimal : 0x00000000		

Tabel 2.2 Observasi Instruksi Branch & Memori

Instruksi	Alamat	Branch	Cabang diambil? (ya/tidak)	PC berikutnya
Bne t0, t1, gagal	0x0000000C	1	Ya	0x00000018 (gagal)

Apakah masukan multiplexer berasal dari "hasil ALU" atau ada dari memori? Jelaskan alasannya!

Pada instruksi lw, multiplexer memilih data dari memori. Karena, tujuan instruksi lw (Load Word) adalah memuat isi data yang tersimpan dalam memori data ke dalam register tujuan (rd), bukan menyimpan hasil perhitungan aritmetika dari ALU.

Soal 3: Lokalitas Spasial dan Ukuran Line

Tabel 3.1 Observasi Ukuran Line (Data Cache**)**)

Word/line	Access Count	Hits	Misses	Hit Ratio	Miss Ratio
1	32	0	32	0.00 (0%)	1.00 (100%)
GANTI WORDS/LINE SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
2	32	16	16	0.50 (50%)	0.50 (50%)
GANTI WORDS/LINE SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
4	32	24	8	0.75 (75%)	0.25 (25%)
GANTI WORDS/LINE SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
8	32	28	4	0.875 (87.5%)	0.125 (12.5%)

Jelaskan mengapa **hit ratio** meningkat seiring bertambahnya **word/line**, dan sebutkan jenis miss yang terjadi pada program ini beserta alasanya!

Karena **lokalitas spasial**. Saat terjadi miss, cache langsung memuat satu baris penuh berisi beberapa data berurutan. Jadi, saat data di sebelahnya dibaca, data tersebut sudah ada di cache (hit). Semakin besar Words/Line, semakin banyak data berdekatan yang ikut dimuat.

Compulsory Miss (Cold Miss), yaitu *miss* yang wajar terjadi karena data diakses untuk pertama kalinya dan cache masih kosong.

Soal no 4 : Lokalitas Temporal dan kapasitas cache

Tabel 4.1 Observasi Kapasitas Cache

Lines	Kapasitas(byte)	Access count	Hits	Misses	Hit ratio
2	32	80	48	32	0.60 (60%)
GANTI LINES SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
4	64	80	48	32	0.60 60%)
GANTI LINES SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
8	128	80	64	16	0.80 (80%)
GANTI LINES SESUAI KETENTUAN, LALU TEKAN RESET SEBELUM LANJUT					
16	256	80	64	16	0.80 (80%)

Jelaskan mengapa **hit ratio** pada **Lines = 4, 8, dan 16** tidak berubah, dan jenis miss apa yang tersisa!

Pada Lines = 2, cache hanya bisa menampung 32 byte, sedangkan ukuran array 64 byte. karena itu, data tidak bisa masuk semua dan harus bergantian, sehingga sering terjadi miss. Sedangkan pada Lines = 4, 8, dan 16, semua data sudah bisa masuk ke cache. Karena itu, hit ratio-nya sama. Pada putaran pertama terjadi 16 kali miss karena data baru pertama kali diakses. Selanjutnya, data sudah ada di cache sehingga akses berikutnya menjadi hit.